

Optimización multiobjetivo para compresión en audio

- **Autor:** Esp. Prof. Alejandro Lloveras
- **Carrera:** Maestría en Inteligencia Artificial
- **Institución:** Facultad de Ingeniería (UBA)
- **Materia:** Algoritmos Evolutivos
- **Docente:** Prof. Miguel Augusto Azar

En este proyecto se propuso optimizar la compresión de archivos de audio WAV a MP3 en función de los algoritmos *Perceptual Evaluation of Audio Quality* (PEAQ) y *Distortion Index* (DI), métricas de audiopercepción que se basan en un modelo del oído humano. El objetivo fue reducir el tamaño de archivo sacrificando la menor calidad perceptual de audio.

► [Repositorio GitHub](#)

Definición del problema

La utilización de algoritmos de compresión con pérdida (*lossy* en inglés) para archivos de multimedia es compleja y requiere la configuración de múltiples parámetros diferentes que pueden dar lugar a resultados diversos. En el caso del audio en particular, la compresión utilizada para archivos MPEG-2 Audio Layer III (popularmente conocidos como MP3) ofrece 6 parámetros configurables con 520.290 combinaciones posibles. De la adecuada selección de estos, dependerá la calidad de la compresión. Por otro lado, la respuesta en frecuencia del oído humano es extremadamente compleja y presenta un comportamiento no lineal.

Las curvas de Fletcher-Munson, también conocidas como curvas isofónicas, describen cómo son percibidas las frecuencias a distintos niveles de volumen. Como puede apreciarse en la Figura 1, el comportamiento del oído varía en función de la naturaleza de la fuente (frecuencia, ancho de banda, timbre, componentes armónicos, sonoridad). Es evidente por tanto, que para lograr una compresión ideal para una conversación no se requerirán los mismos parámetros que para un archivo de música.

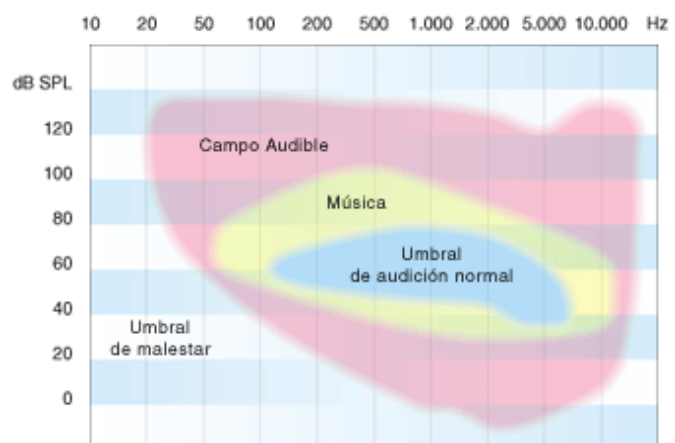


Fig 1. Curvas isofónicas de la audición humana.¹

Para lograr encontrar la combinación de parámetros óptima que minimice el tamaño del archivo MP3 con la menor pérdida de calidad de audio, es necesario poder cuantificar adecuadamente la percepción del sistema auditivo humano, por lo que es necesario construir un modelo matemático que lo emule adecuadamente. Para el presente trabajo, se

¹ Imagen tomada del sitio web <http://elruido.com/divulgacion/curso/umbrales.htm>

utilizaron la métrica PEAQ (*Perceptual Evaluation of Audio Quality*)² y el índice de distorsión IM (*Improved Metric of Informational Masking*)³. Estas métricas abarcan un rango de 0 a -4, donde 0 es “máxima calidad” y -4 “pérdida extrema”. Particularmente, IM puede tomar valores aún menores a -4 indicando un deterioro mayor.

Implementación

El objetivo principal es encontrar un conjunto óptimo de parámetros de codificación de audio que minimicen el tamaño del archivo resultante y el tiempo de procesamiento, y al mismo tiempo maximicen la calidad de audio (medida por PEAQ y un índice de distorsión). Se trata por tanto de un problema de optimización multiobjetivo. Inicialmente se definieron direcciones de optimización con valores extremos: 1 para maximizar o -1 para minimizar.

Librerías de terceros

Para la implementación de la solución fue necesario instalar varias librerías de terceros. Se utilizó el [framework FFmpeg](#) de software libre para la conversión de archivos multimedia (controlado a través de la [librería “ffmpeg-python”](#)).

Para la emulación del oído humano y la medición de calidad de audio (PEAQ e IM) se trabajó con el [plugin “GST_Peaq”](#) para el [framework GStreamer](#), accediendo a través de comandos de terminal.

Para la implementación del algoritmo genético se utilizó el [framework DEAP](#), un marco de computación evolutiva para el prototipado rápido y la prueba de ideas.

Desarrollo

El paradigma de programación utilizado fue el desarrollo modular. Inicialmente se desarrollaron los módulos de compresión de audio y cálculo de métricas:

- ***compressor.py***: Se encarga de la compresión de audio real de WAV a MP3 usando `ffmpeg-python`, aplicando los parámetros especificados por el algoritmo evolutivo.
- ***peaq.py***: Proporciona una interfaz a la herramienta externa de línea de comandos `peaq`, utilizada para mediciones objetivas de la calidad de audio perceptual. También gestiona las variables de entorno para una correcta ejecución de la herramienta.

A su vez, estos módulos son accionados por:

- ***orchestrator.py***: Actúa como el centro neurálgico del proceso de evaluación. Llama a `compressor.py` para comprimir el audio y a `peaq.py` para evaluar la calidad del archivo comprimido. También mide el tamaño del archivo y el tiempo total de procesamiento.

² Kabal, P. (2002). *An Examination and Interpretation of ITU-R BS.1387: Perceptual Evaluation of Audio Quality*. McGill University. <https://www.mmsp.ece.mcgill.ca/Documents/Reports/2002/KabalR2002v2.pdf>

³ Delgado, P. M., & Herre, J. (2023). *An improved metric of informational masking for perceptual audio quality measurement*. arXiv preprint arXiv:2307.06656. <https://arxiv.org/abs/2307.06656>

Los archivos multimedia son almacenados en la carpeta `"media"`. Se trabajó con archivos WAV de entrada y los archivos MP3 de salida (para el ciclo genético sólo se almacenan temporalmente).

A continuación, se desarrolló la estructura base para el ciclo evolutivo, utilizando el algoritmo NSGA-II (*Non-dominated Sorting Genetic Algorithm II*), para optimizar los parámetros de compresión de audio configurados para FFmpeg. En el script `evolution.py` se definen los genes (parámetros de compresión de audio) utilizando DEAP y se orquesta la evaluación de cada individuo, enviando estos parámetros a `orchestrator.py`.

Dada la enorme cantidad de datos generados en cada evaluación, se detectó la necesidad de generar archivos de logging, definidos a través del script `logger.py` y almacenados en la carpeta `"logs"`.

Cada uno de los individuos generados durante el ciclo evolutivo es almacenado en un archivo CSV dentro del directorio `"history"`, almacenando los parámetros de FFmpeg y los objetivos evaluados. Al finalizar la evolución, se almacena dentro de `"results"` otro CSV con los mejores individuos encontrados con el frente de Pareto, denominado como *Hall of Fame* (HOF) en DEAP.

Finalmente, se evalúan los resultados obtenidos con la ayuda de un cuaderno Jupyter denominado `graph.ipynb` que facilita el análisis y visualización del ciclo de entrenamiento.

Algoritmo genético

El NSGA-II (*Non-dominated Sorting Genetic Algorithm II*) es un algoritmo genético multiobjetivo popular, diseñado para resolver problemas con múltiples funciones objetivo que a menudo están en conflicto entre sí. Su objetivo principal es encontrar un conjunto de soluciones que representen el frente de Pareto óptimo, es decir, soluciones donde no se puede mejorar una función objetivo sin empeorar al menos otra.

Parámetros Genéticos (Genes)

Cada "individuo" en la población representa un conjunto de parámetros FFmpeg y consta de los siguientes genes:

- **Tasa de muestreo de audio (`ar`)**: Un índice que se mapea a una lista predefinida de tasas de muestreo comunes (8000 Hz a 48000 Hz).
- **Profundidad de bits —*Bit Depth*— o Formato de muestreo (`sample_fmt`)**: Un índice que se mapea a formatos de muestra como 's16p', 's32p' o 'fltp'.
- **Nivel de compresión (`compression_level`)**: Un entero entre 0 (mejor calidad/menor compresión) y 9 (mayor compresión).
- **Reservorio (`reservoir`)**: Un booleano (0 o 1) para activar o desactivar el control de bits del reservorio de LAME.
- **Modo de codificación (`encoding_mode`)**: Un índice que selecciona entre:
 - 0: `audio_bitrate` (CBR - Constant Bit Rate)
 - 1: `aq` (VBR Quality - Variable Bit Rate)
 - 2: `abr` (Average Bitrate)

- **Valor del modo:** Un valor flotante que se interpreta como bitrate (en kbps) para CBR/ABR o calidad VBR (0-9).

Función de Fitness

La función de fitness, `"evaluate_ffmpeg_params"`, realiza los siguientes pasos para cada individuo:

- **Generación de Parámetros FFmpeg:** Los genes del individuo se decodifican y se transforman en un diccionario de parámetros para FFmpeg. Se aplican restricciones y mapeos para asegurar la validez de los parámetros.
- **Ejecución de la Evaluación:** Se invoca una función `"evaluate"` (definida en `orchestrator.py`) que ejecuta FFmpeg con los parámetros generados y mide métricas clave del archivo de audio resultante:
 - `size`: Tamaño del archivo en kilobytes.
 - `peaq`: Puntuación PEAQ (*Perceptual Evaluation of Audio Quality*).
 - `im`: Índice de distorsión.
 - `time`: Tiempo de procesamiento.

Problemas durante el desarrollo

Parámetros de FFmpeg

Dado que la documentación de la librería "ffmpeg-python" era bastante incompleta, fue necesario realizar numerosas pruebas a través de la línea de comandos para determinar el rango y tipo de parámetros configurables para la compresión de WAV a MP3.

Tiempo de procesamiento

Si bien la conversión de archivo realizada por FFmpeg es relativamente rápida, al realizar entrenamientos largos con miles de individuos, los tiempos de procesamiento acumulados para múltiples archivos pueden ser considerables, alargando los entrenamientos por múltiples horas. Por ejemplo, para una canción estándar de 4 min de música (archivo .wav de 70 MB con 24 bits de profundidad y frecuencia de muestreo 44,1 kHz) el ciclo completo de compresión y análisis se realizaba en 11,3 segundos de media. Realizando un entrenamiento de tan sólo 2000 individuos, el tiempo se elevaría a más de 6 horas.

Para reducir el tiempo de procesamiento de audio al máximo se decidió utilizar la implementación "simple" de PEAQ (5,3 seg), en lugar de la "advanced" (43,8 seg), ya que incrementaba el tiempo de análisis en 8,5x veces. De igual modo, se generaron pequeños fragmentos de música con duraciones de 5 y 10 seg para testeo durante la etapa de desarrollo. Para el entrenamiento real, se generaron fragmentos representativos de música real de 15 seg de un ensamble musical (grabación de homestudio) y de 11 seg de una orquesta (grabación de estudio profesional). Estos fragmentos fueron seleccionados cuidadosamente para que contuvieran un rango armónico y dinámico amplio.

Calidad de audio

La calidad estándar de CD de audio de 44,1 kHz y 24 bits es ampliamente utilizada por toda la música comercializada. Fue extremadamente difícil conseguir archivos sin pérdida (formato FLAC o WAV) que permitieran medir efectivamente el efecto de la compresión aplicada. El fragmento de 15 segundos (con 44,1kHz y 24bits) era un material de autoría propia grabado con anterioridad.

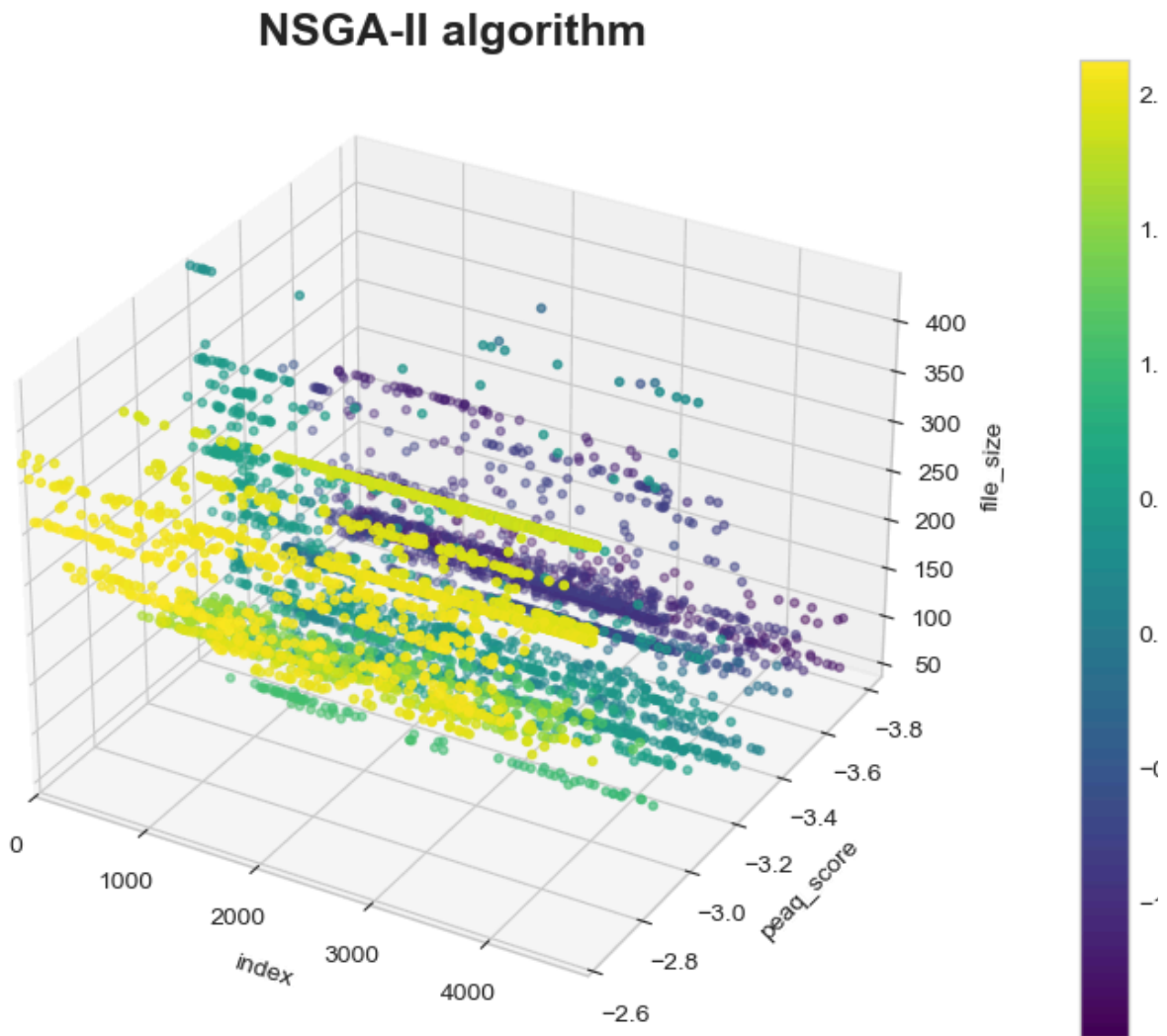


Fig 2. Evolución temporal de PEAQ y compresión (relacionado con *Fitness*).

El concierto de cuerdas (con 96kHz y 24bits) —fragmento de 11 segundos— fue obtenido posteriormente en un repositorio de música libre de derechos de autor en formato FLAC. Se decidió trabajar con este fragmento por ser el de mayor calidad, permitiendo analizar el efecto de la reducción de la frecuencia de muestreo a 48 kHz y compararlo con el de 44,1kHz.

Ponderación

Si bien inicialmente se trabajó únicamente definiendo la dirección de optimización, pronto se observó que el algoritmo evolutivo alcanzaba siempre la optimización reduciendo la frecuencia de muestreo, ignorando los demás objetivos, aún cuando esto perjudicara las métricas de calidad.

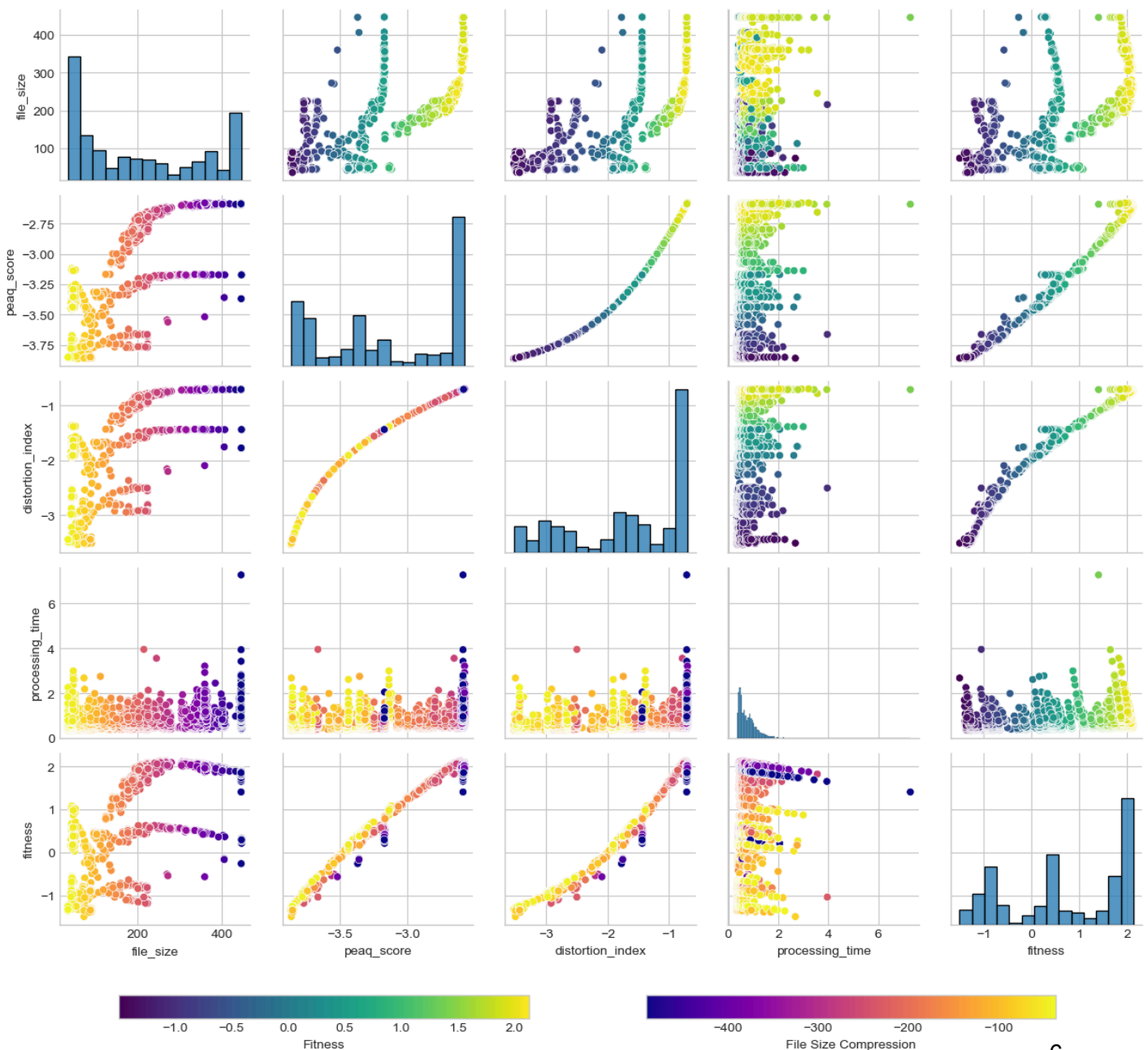
Se decidió entonces asignar distintas ponderaciones a cada objetivo para darle mayor importancia a la métrica PEAQ que al tamaño de archivo, dado que una compresión extrema no era deseable.

Las ponderaciones fueron ajustadas manualmente hasta obtener valores “razonables” (según criterio de especialista) de sample rate y bit rate, quedando definidas de la siguiente forma:

1. **100% peaq**: métrica de calidad,
2. **50% size**: tamaño del archivo en kilobytes,
3. **30% im**: índice de distorsión,
4. **5% time**: tiempo de procesamiento.

Se le dió poca importancia a IM por estar mayormente correlacionado con PEAQ (exceptuando casos puntuales). Por su parte, al objetivo del tiempo de procesamiento se le asignó un peso mínimo para que, en caso de obtener dos individuos con valores de calidad y peso similares, se escogiera aquel que requiriera menor procesamiento.

Multi-objective optimization (NSGA-II)



Normalización

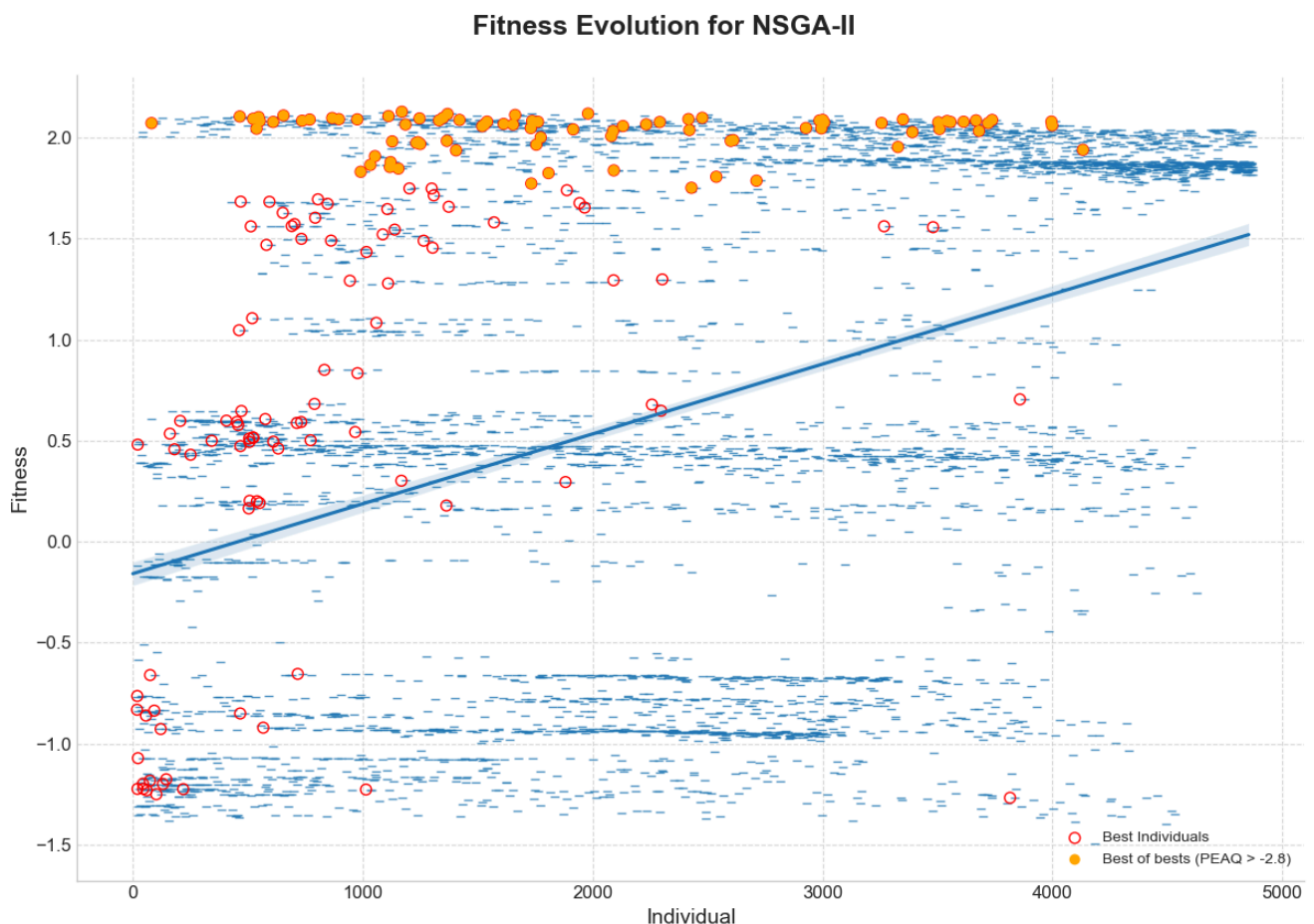
Inicialmente se pensó que DEAP normalizaba los objetivos, pero luego se detectó la necesidad de aplicar normalización a los valores de la función fitness por ser de magnitudes diferentes: tamaño de archivo en kilobytes, procesamiento en milisegundos y métricas de calidad en el rango 0 a -4.

Se aplicó normalización z-score utilizando valores de media y desviación estándar precalculados a partir de los datos almacenados durante las ejecuciones previas (en torno a 15.000 individuos).

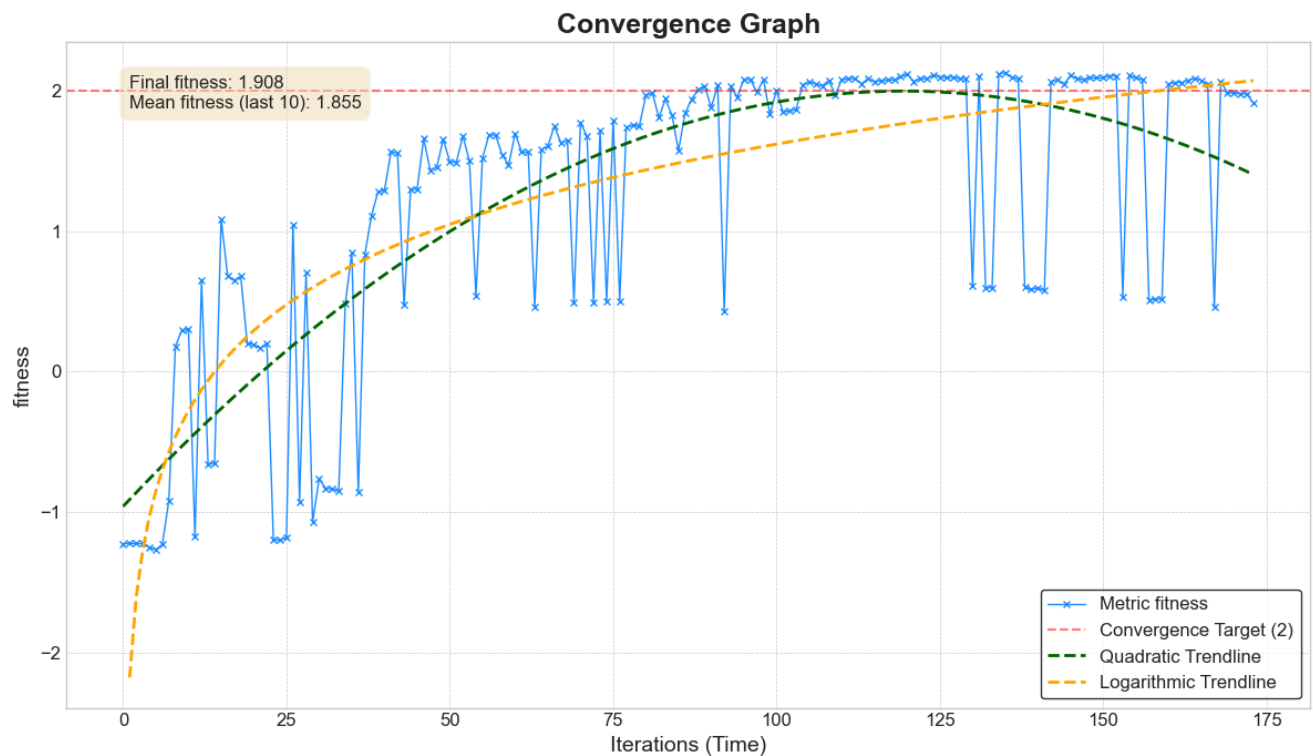
Gráfico de convergencia

Dada la gran cantidad de individuos analizados y la sensibilidad de las métricas objetivo a pequeños cambios en la configuración de los parámetros de compresión de FFmpeg, la métrica fitness presentaba gran volatilidad durante el entrenamiento, lo que dificultaba la visualización de la convergencia.

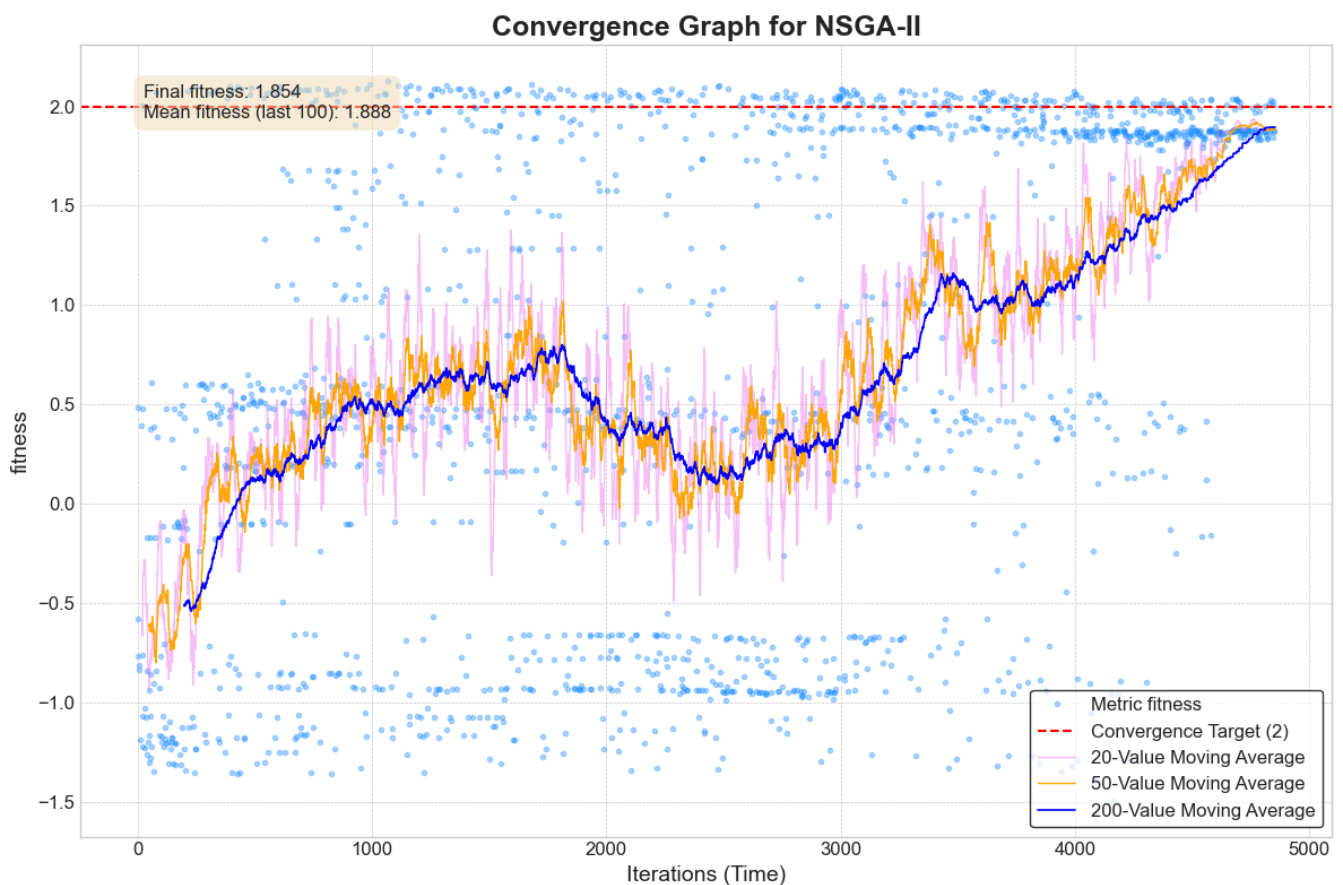
Se decidió entonces visualizar la evolución a través de la variación del fitness promedio. En una primera instancia se ajustó una curva de tendencia para observar si la pendiente era positiva. Se destacaron además los individuos pertenecientes al HOF y, entre esos, se seleccionaron aquellos que poseían mejores métricas PEAQ (umbral -2,8).



Se exploró también observar sólo los individuos del HOF, trazando líneas de tendencia cuadrática y logarítmica como referencia para observar su evolución.



Finalmente se decidió trabajar con un set de 3 medias móviles de 20, 50 y 200 individuos, para observar la tendencia general y la volatilidad de medio y corto plazo.



Operadores genéticos (NSGA-II)

A través de varias ejecuciones experimentales con el algoritmo NSGA-II, se exploraron distintas configuraciones de operadores genéticos (tamaño de población, número de generaciones, probabilidad de cruce y de mutación) con el objetivo de determinar aquellos hiperparámetros que maximizaran la función de fitness (métricas multiobjetivo normalizadas y ponderadas).

La estrategia aplicada consistió en variar uno a uno los parámetros operadores genéticos y parámetros de población de forma ascendente y descendente, para evaluar su impacto en el proceso de optimización.

Configuración inicial

NSGA-II	Base	Mutación ↑	Mutación ↓	Población ↓	Población ↑	Generaciones ↑	Cruce ↑	Mix
Prob. de Cruce	80%	80%	80%	80%	80%	80%	95%	95%
P. de Mutación	10%	20%	5%	10%	10%	10%	5%	1%
Población	50	50	50	25	100	50	50	50
Generaciones	50	50	50	50	50	100	50	100
Individuos	2500	2500	2500	1250	5000	5000	2500	5000

Resultados obtenidos

	Base	Mutación ↑	Mutación ↓	Población ↓	Población ↑	Generaciones ↑	Cruce ↑	Mix
Fitness medio	1,759	0,840	0,310	1,038	0,302	1,915	0,735	1,888
Ganadores	115	205	91	103	149	118	205	174
Óptimos	62	101	37	65	72	57	94	83
Tasa de éxito	54%	49%	41%	63%	48%	48%	46%	48%
Compresión	-96,648%	-96,646%	-96,501%	-96,678%	-96,596%	-96,625%	-96,767%	-96,856%
PEAQ Score	-2,628	-2,635	-2,639	-2,636	-2,628	-2,635	-2,645	-2,648
Índice de distorsión (IM)	-0,745	-0,753	-0,758	-0,754	-0,745	-0,753	-0,764	-0,768

2º Mejor

Mejor

Puede verse en la tabla que, gracias al ajuste de hiperparámetros, se logró alcanzar una mayor comprensión con métricas de calidad similares. En particular, se encontró que:

- Para un mismo número de individuos, al incrementar el número de generaciones se obtiene mayor mejoría que acrecentando la población.
- Al incrementar la probabilidad de mutación, la exploración se volvía más errática impidiendo la mejora.

- Si se incrementaba la probabilidad de cruce, se lograban mejores resultados en la compresión de audio (por explorar distintas combinaciones de parámetros de FFmpeg).
- Se pueden evitar estancamientos en máximos locales disminuyendo la probabilidad de mutación.
- Al combinar mayor probabilidad de cruce y menor de mutación, la evolución era más lenta pero controlada (ergo, predecible).

Exploración de otros algoritmos

Posteriormente, se contrastó el resultado de NSGA-II contra otros algoritmos estándar para problemas multi-objetivos: NSGA-III, SPEA2, PAES y MOEA-D.

Para su ejecución se realizaron las modificaciones mínimas necesarias para poder implementar el nuevo algoritmo, intentando conservar similares números de individuos y operadores genéticos.

Configuración inicial

	NSGA-II	NSGA-III	SPEA2	PAES	MOEA/D
Prob. de Cruce	95%	95%	95%	N/A	95%
P. de Mutación	1%	1%	1%	50%	1%
Población	50	50	50	50	165
Generaciones	100	100	100	5000	25
Individuos	5000	5000	5000	5001	4291

Resultados obtenidos

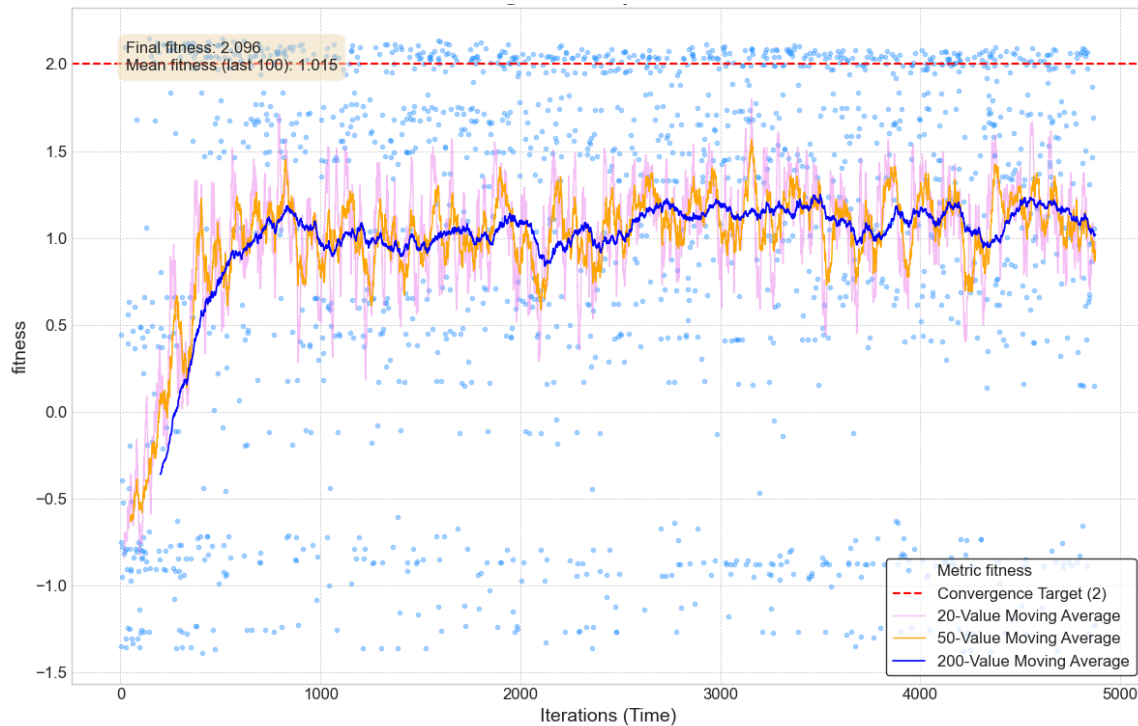
	NSGA-II	NSGA-III	SPEA2	PAES	MOEA/D
Fitness medio	1,888	1,015	0,766	0,273	1,551
Ganadores	174	292	220	17	132
Óptimos	83	155	125	5	75
Tasa de éxito	48%	53%	57%	29%	57%
Tamaño (kB)	267,83	258,68	276,63	417,06	251,62
Compresión	-96,856%	-96,964%	-96,753%	-95,105%	-97,047%
PEAQ Score	-2,648096	-2,651303	-2,639016	-2,5902	-2,7136
Índice de distorsión (IM)	-0,767831	-0,771335	-0,757792	-0,704	-0,79364

Mejor

2° Mejor

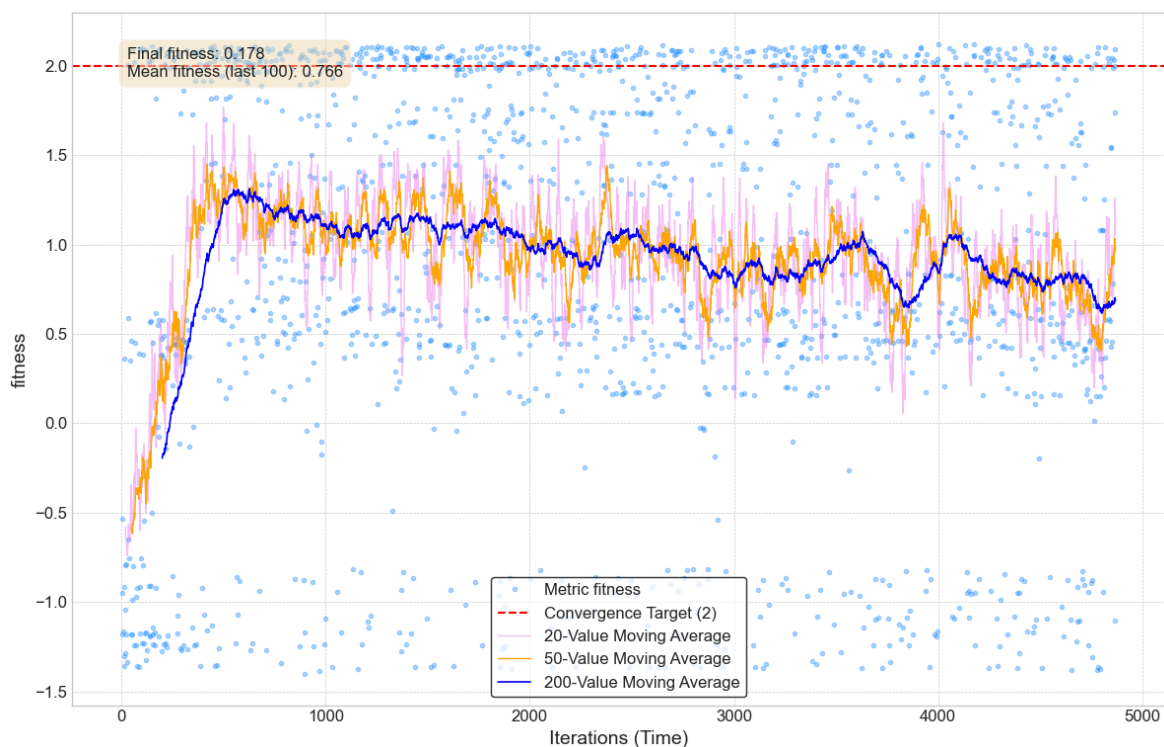
NSGA-III

Como puede verse, logró rápidamente una mejora mayor a NSGA-II, pero luego se estancó en esa región siendo prácticamente incapaz de mejorar durante el resto del ciclo.



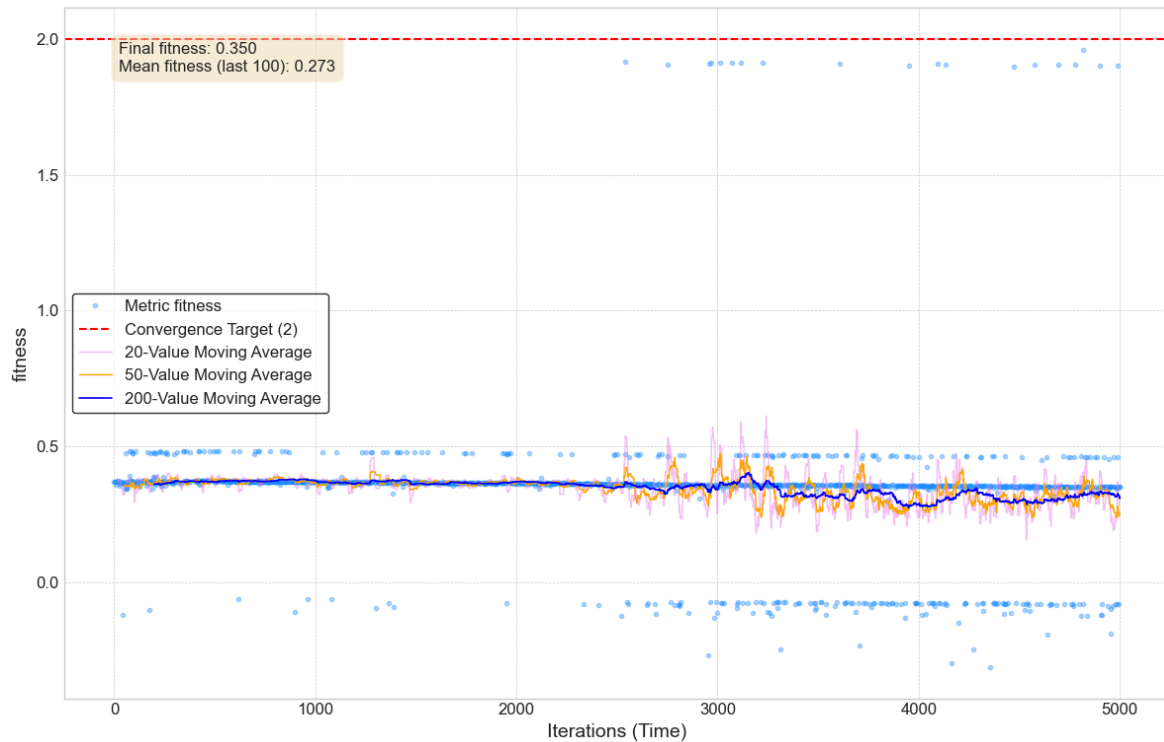
SPEA2

De modo similar, SPEA2 alcanzó una rápida mejora (cerca del individuo 500) para luego empeorar progresivamente, generando un mayor número de individuos en la región central (fitness de media 0,5).

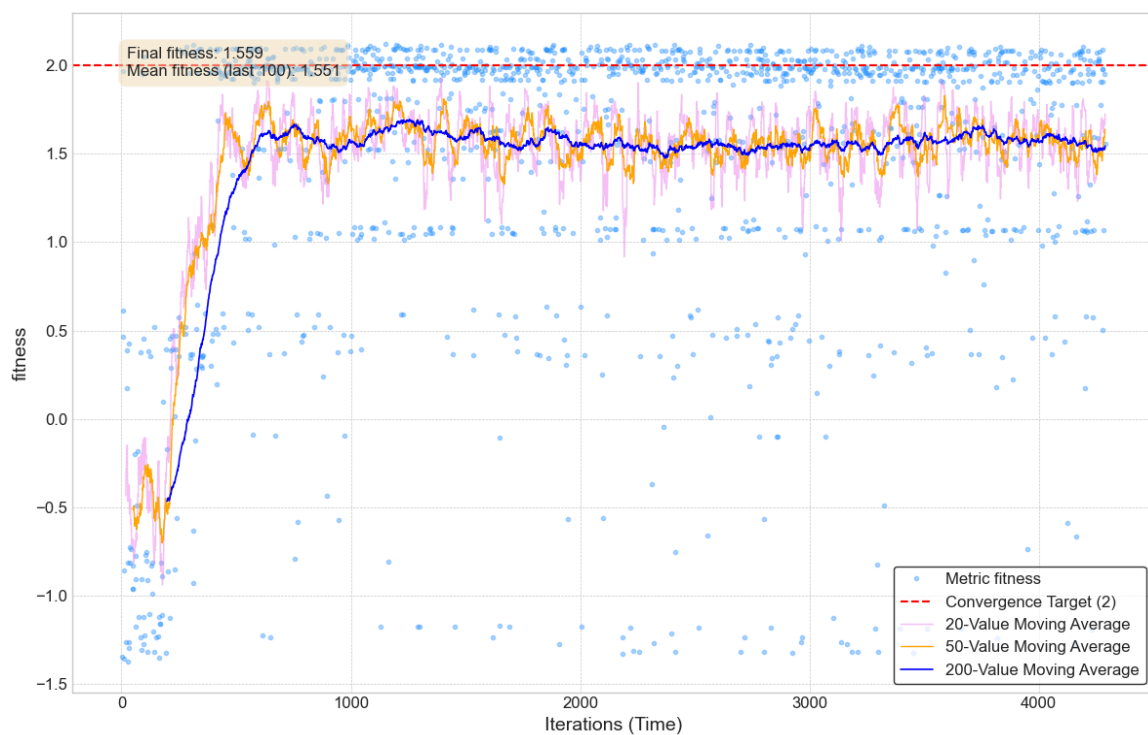


PAES

PAES fue el que ofreció los peores resultados, ya que se pudo ver que es muy dependiente de las condiciones iniciales, que prácticamente determinan el resultado final, sin obtener mejoras significativas durante el ciclo genético. Fue necesario correrlo varias veces para lograr una inicialización “conveniente” (la aquí presentada) y sin embargo los individuos fueron de mucho peor calidad que con los otros algoritmos.



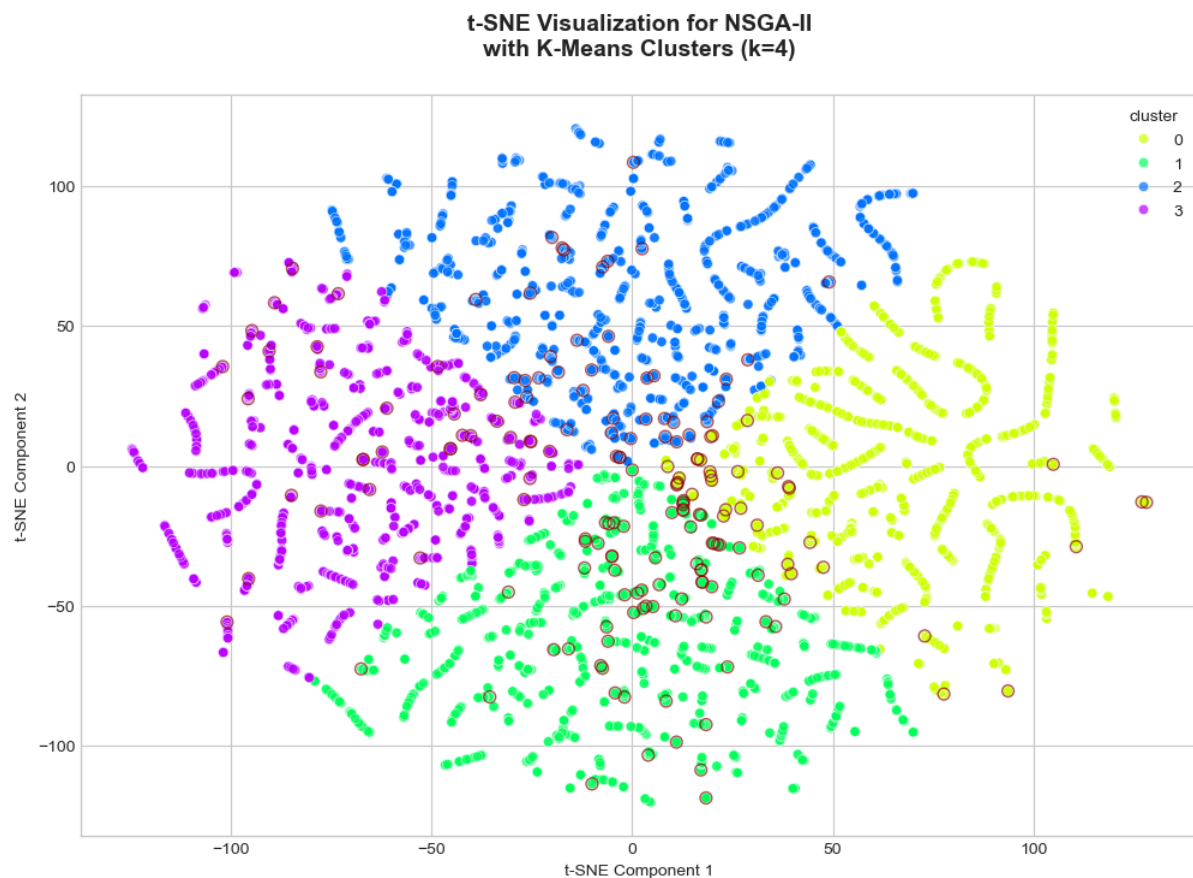
MOEA/D



MOEA/D fue bastante más complejo de configurar y no escala muy bien, ya que pequeñas modificaciones de los hiperparámetros provocan un crecimiento exponencial del número total de individuos, lo que lo vuelve difícil para entrenar en un tiempo razonable. A pesar de esto, se observa que ofreció mejores resultados que los otros algoritmos alternativos, al lograr una rápida mejora (cerca del individuo 500), para luego mantenerse con valores medios de *Fitness* de 1,5, pero generando muchos individuos óptimos (cercanos al target).

Clusterización y análisis de parámetros

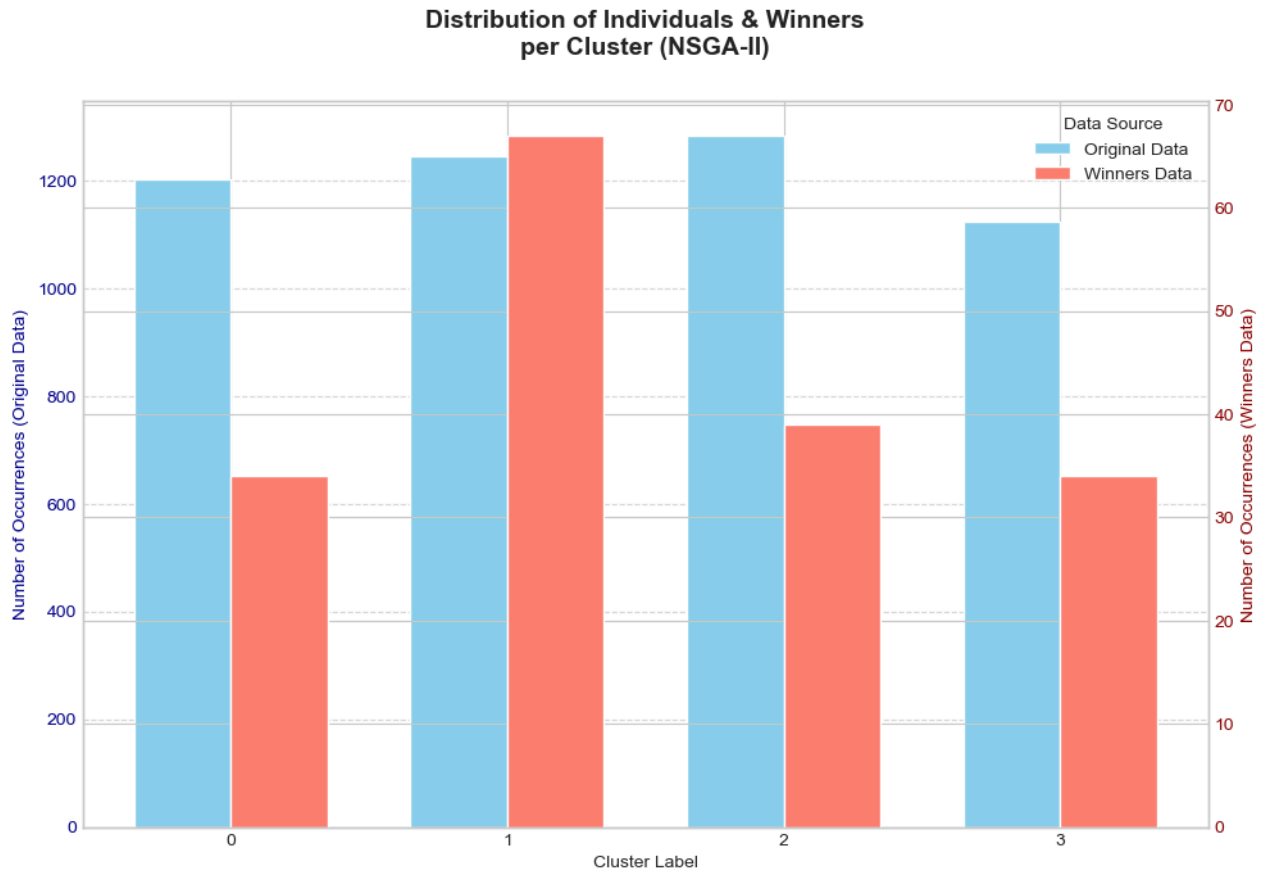
Se aplicó t-SNE, utilizando el índice de Silhouette, para identificar clusters dentro de los individuos del HOF. Además se destacaron con un redondel, aquellos individuos que cumplen la condición establecida anteriormente ($PEAQ > -2,8$).



De este modo, se detectan los clusters con mayor número de individuos “óptimos”, y se realiza un análisis de los parámetros de FFmpeg que comparten entre sí.

Hall of Fame (174 individuos)

- **Reservoir:** se encuentra casi siempre activado (88% de los individuos). Esto sugiere que el control de bits del reservorio es beneficioso para la optimización de fitness.
- **Frecuencia de muestreo (ar):** Mayormente 44,1 kHz (71%), frecuencia estándar para calidad de CD. Demuestra que los resultados son viables para la industria.



- **Modo de codificación:**
 - **Average Bit Rate (ABR):** activado para la mayoría de los individuos (72%). Lo cual indica que es el mejor modo de codificación (*encoding mode*) para los objetivos planteados.
 - **Variable Bit Rate (VBR):** un 15% de los individuos ganadores utilizaron tasa de bits variable.
 - **Constant Bit Rate (CBR):** mientras que el 13% restante la mantuvo constante.
- **Tasa de bits para ABR/CBR (*b:a*):** valores continuos en el rango de 130-170 kbps.
- **Calidad de audio VBR (*aq*):** mayor fitness para valores 2, 4 y 5.
 - Valores medios (4 y 5) muestran un gran balance entre calidad y compresión.
 - Sorprendentemente, un valor extremadamente bajo como lo es 2 también logró un buen fitness alto (1,93686). Esto podría indicar que, para ciertas configuraciones, una calidad VBR más "baja" (que podría significar menor bitrate promedio) aún puede producir resultados perceptualmente muy buenos.
- **Nivel de compresión (*compression_level*):**
 - La compresión máxima (nivel 9) es bastante frecuente (36% de los individuos), apareciendo en todos los clústeres, así como los valores cercanos 7 y 8 (juntos

describen al 60% de los individuos del HOF). Esto indica que muchas veces el algoritmo utilizó este parámetro para reducir el tamaño.

- La compresión media (nivel 4) fue utilizada para el 21% de los individuos.
- Valores entre 0 y 3, que indican compresión mínima, fueron utilizados tan sólo el 13% de las veces.
- **Profundidad de bits / Bit Depth (*sample_fmt*)**: 's16p' es el formato más común en la mayoría de los individuos (58%). Ampliamente compatible y de calidad estándar.

Es importante destacar también aquellos parámetros que no fueron ganadores:

- ✗ El hecho de que **ningún individuo del HOF utilice 48 kHz** de muestreo, evidencia que el incremento en resolución aportado **no se encuentra justificado a nivel perceptual**. Siendo esta una discusión habitual entre audiófilos y productores.
- ✗ **Nivel de compresión**: en ningún caso se utilizó el nivel 5 y en poca medida el 6.

Cluster más exitoso (67 individuos)

- **Frecuencia de muestreo (*ar*)**: 44,1 kHz para todos.
- **Reservoir**: generalmente activado (85%). Ahorro de bits durante pasajes simples.
- **Modo de codificación**:
 - **Average Bit Rate (ABR)**: 80% de los individuos del cluster utilizaron codificación promedio.
 - **Variable Bit Rate (VBR)**: un 16% utilizaron tasa de bits variable.
 - **Constant Bit Rate (CBR)**: tan sólo el 4% trabajó con tasa constante.
- **Tasa de bits para ABR/CBR (*b:a*)**: en torno a 148 kbps (80%).
- **Calidad de audio VBR (*aq*)**: *mayormente con valores medios (6 y 5)*.
- **Profundidad de bits / Bit Depth (*sample_fmt*)**: 's16p', representa un número entero con signo de 16 bits, planar (66%).
- **Nivel de compresión (*compression_level*)**:
 - 9, compresión alta (40%),
 - 4, compresión media (30%).

Individuo óptimo

Parámetros FFmpeg:

{*ar*: '44100', *sample_fmt*: 's32p', *compression_level*: '4', *reservoir*: '0', *b:a*: '190k'}

- **Frecuencia de muestreo / Frequency Rate (*ar*)**: 44,1 kHz.
- **Profundidad de bits / Bit Depth (*sample_fmt*)**: 's32p', número entero de 32 bits con signo, planar.

- **Modo de codificación:** Constant Bit Rate (CBR).
- **Tasa de bits / Audio Bitrate (*b:a*):** 190 kbps.
- **Nivel de compresión (*compression_level*):** 4, compresión media.
- **Calidad de audio VBR (*aq*):** desactivado.
- **Reservoir:** desactivado.

Con estos parámetros se logró durante el ciclo evolutivo un valor de **Fitness** de **2,127855**, compuesto por **PEAQ** de **-2,613**, **IM** de **-0,728** y una **compresión efectiva** del **-96,859%**.

Como última instancia, se procesó un fragmento musical completo con los mismos parámetros, para verificar si eran generalizables a la pista completa. Partiendo de un **archivo WAV de 22,3 MB**, se lo comprimió a **formato MP3 de 698 kB**, una **reducción del -96,867%**, con métricas **PEAQ** en **-2,593** e **IM -0,707**. Se verifica que se obtienen valores similares al del individuo óptimo. Procesar 4 min de música tomó tan sólo 5,24 segundos.

A modo informativo, al comparar auditivamente ambos archivos —con audífonos profesionales— no es perceptible una diferencia de calidad significativa ni pérdida de frecuencias.

Conclusiones

- 1) La calidad de audio CD (44,1 kHz) y el formato estándar (s16p) son óptimos.
- 2) Valores de bitrate altos (entre 130 a 170 kbps) tienden a alcanzar *fitness* elevados.
- 3) ABR es el modo de codificación dominante (72% en el Hall of Fame, 80% en el clúster más exitoso), lo que sugiere que permite el mejor balance entre tamaño de archivo y calidad percibida para los objetivos planteados.
- 4) Sin embargo, puede obtenerse excelentes resultados con VBR ajustado en una calidad media, o incluso con CBR, estableciendo un nivel de compresión intermedio.
- 5) Si se desea maximizar la reducción del tamaño de archivo, es preferible una compresión agresiva (nivel 9). Por el contrario, si se desea preservar la calidad, es preferible valor medio (nivel 4).
- 6) La activación del "reservoir" para el control de bits fue una característica clave para la eficiencia (88% en el Hall of Fame, 85% en el clúster más exitoso), dado que permite ahorros de bits en pasajes más simples del audio.

Paradójicamente, el individuo seleccionado como "óptimo" no utiliza ninguno de los parámetros más comunes encontrados en el HOF. Esto sugiere que, si bien ABR es generalmente preferido, un CBR con una tasa de bits relativamente alta y un formato de muestra de mayor profundidad puede lograr una calidad perceptual superior junto con una compresión muy alta.